Lectures32-33_MA5104

18 October 2026 10:59

Lectures32
-33_MA5...

MA 5104: Statistical Inference and Learning

Module 4: Learning
Lectures 32-33: Tree-Based Methods

Dr. Rakhi Singh
Department of Mathematics
IIT Madras

Fall 2026

1 / 30

Introduction

Tree-based methods work very well for both regression and classification problems.

Tree-based methods involve *stratifying* or *segmenting* the predictor space into a number of simple regions.

In order to make a prediction for a given observation, we typically use the mean or the mode response value for the training observations in the region to which it belongs.

Since the set of splitting rules used to segment the predictor space can be summarized in a tree, these types of approaches are known as *decision tree methods*.

Tree-based methods are simple and useful for interpretation.

A single tree is often not competitive in terms of prediction accuracy.

That's why we will also talk about *bagging*, *random forests*, boosting.

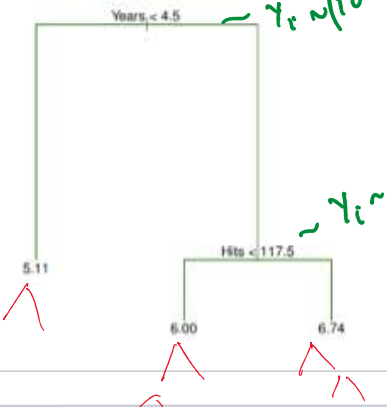
2 / 30

The Basics of Decision Trees – Regression

We use the *Hitters* data set to predict a baseball player's salary based on *Years* (the number of years that he has played in the major leagues) and *Hits* (the number of hits that he made in the previous year). Missing Salary values are removed and salary is log-transformed.

Important things to consider:

- split variable
- split value
- root node
- terminal nodes or leaves
- prediction on a new test point



3 / 30

In terms of regions

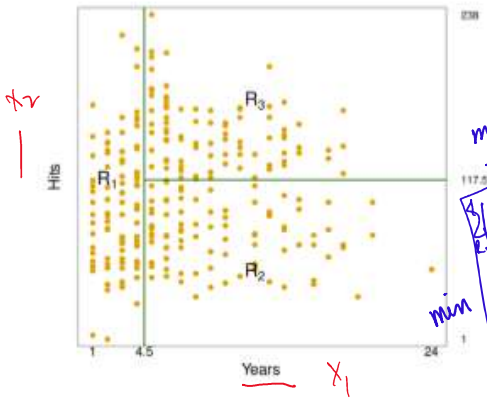


FIGURE 8.2. *The three-region partition for the* Hitters *data set from the regression tree illustrated in Figure 8.1.*

4 / 30

Basics continued

Regions R1, R2, and R3 are known as *terminal nodes* or *leaves* of the tree. Decision trees are typically drawn upside down, in the sense that the leaves are at the bottom of the tree.

The points along the tree where the predictor space is split are referred to as internal nodes. The two internal nodes are indicated by the text *Years* < 4.5 and *Hits* <117.5.

Years is the most imp. factor in determining Salary (why?), and players with less experience earn lower salaries than more experienced players.

Hits play no role in salary of inexperienced players.

But among players who have been in the major leagues for five or more years, players who made more hits last year tend to have higher salaries.

Likely an over-simplification of the true relationship; but easier to interpret, and has a nice graphical representation.

5 / 30

Prediction via Stratification of the Feature Space

Roughly speaking, here are two steps:

- We divide the predictor space — that is, the set of possible values for $X_1, X_2, \ldots, X_p$ — into J distinct and non-overlapping regions, $R_1, R_2, \ldots, R_J$.
- For every observation that falls into the region R_j , we make the same prediction, which is simply the mean of the response values for the training observations in R_j.

But how do we find $R_1, R_2, \ldots, R_J$?

In theory, the regions could have any shape. However, we choose to divide the predictor space into high-dimensional rectangles, or boxes.

The goal is to find boxes $R_1, R_2, \ldots, R_J$ that minimize the RSS, given by

$$\sum_{j=1}^{J} \sum_{i \in R_j} (y_i - \hat{y}_{R_j})^2,$$

where $\hat{y}_{R_j}$ is the mean response for the training obs. within the jth box.

Prediction via Stratification of the Feature Space (contd.)

Computationally infeasible to consider every possible partition of the feature space into J boxes.

Instead, use a top-down, *greedy* approach that is known as *recursive binary splitting*.

Top-down because it begins at the top of the tree (at which point all observations belong to a single region) and then successively splits the predictor space; each split is indicated via *two new branches* further down on the tree.

Greedy because at each step of the tree-building process, the best split is made *at that particular step*, rather than looking ahead and picking a split that will lead to a better tree in some future step.

At each split, we select the predictor X_j and the cutpoint s such that splitting the predictor space into the regions $\{X|X_j < s\}$ and $\{X|X_j \geq s\}$ leads to the greatest possible reduction in RSS.

Prediction via Stratification of the Feature Space (contd.)

At each split, we select the predictor X_j and the cutpoint s such that splitting the predictor space into the regions $\{X|X_j < s\}$ and $\{X|X_j \geq s\}$ leads to the greatest possible reduction in RSS.

Consider all predictors $X_1, \ldots, X_p$, and all possible values of the cutpoint s for each of the predictors, and then choose the predictor and cutpoint such that the resulting tree has the lowest RSS, or that, find a j and s that minimize

$$\sum_{i:\, x_i \in R_1(j,s)} (y_i - \hat{y}_{R_1})^2 + \sum_{i:\, x_i \in R_2(j,s)} (y_i - \hat{y}_{R_2})^2,$$

where $\hat{y}_{R_1}$ is the mean response for the training observations in $R_1(j, s)$, and $\hat{y}_{R_2}$ is the mean response for the training observations in $R_2(j, s)$, and

$$R_1(j, s) = \{X|X_j < s\} \text{ and } R_2(j, s) = \{X|X_j \geq s\},$$

are respective regions.

Prediction via Stratification of the Feature Space (contd.)

Finding the values of j and s can be done quite quickly, especially when the number of features p is not too large.

The process, looking for the best predictor and best cutpoint, is repeated in order to split the data further so as to minimize the RSS within each of the resulting regions.

We now split one of the two previously identified regions.

The process continues until a stopping criterion is reached; for instance, we may continue until no region contains more than five observations.

Once the regions $R_1, \ldots, R_J$ have been created, we predict the response for a given test observation using the mean of the training observations in the region to which that test observation belongs.

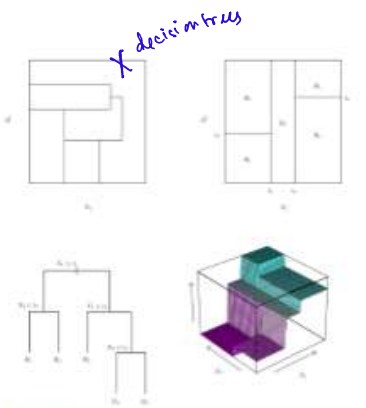


FIGURE 9.2. Top Left: A partition of two-dimensional feature space that could not result from recursive binary splitting. Top Right: The output of recursive binary splitting on a two-dimensional example. Bottom Left: A tree corresponding to the partition in the top right panel. Bottom Right: A perspective plot of the prediction surface corresponding to that tree.

Tree Pruning

The process described above may produce good predictions on the training set, but is likely to overfit the data, leading to poor test set performance.

This is because the resulting tree might be too complex.

A smaller tree with fewer splits might lead to lower variance and better interpretation at the cost of a little bias.

One possible alternative: build the tree only so long as the decrease in the RSS due to each split exceeds some (high) threshold.

This strategy will result in smaller trees, but is too short-sighted since a seemingly worthless split early on in the tree might be followed by a very good split — that is, a split that leads to a large reduction in RSS later on.

Therefore, a better strategy is to grow a very large tree T_0, and then *prune* it back in order to obtain a subtree.

How to prune the tree?

Finding a tree that minimizes the test error using cross-validation? Sure, but an extremely large number of possible subtrees.

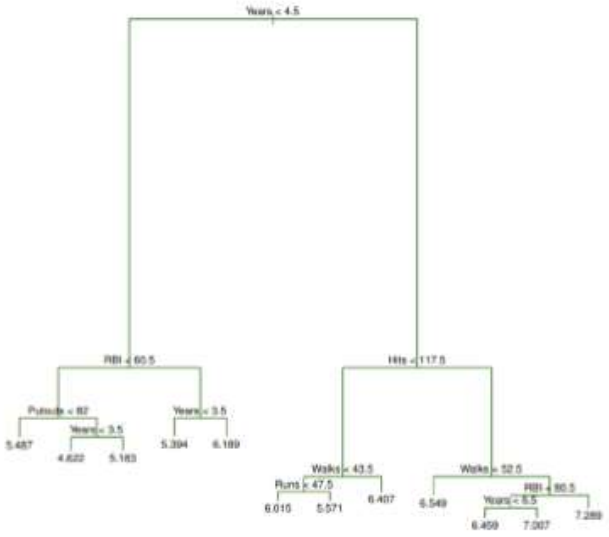
Rather than considering every possible subtree, we consider a sequence of trees indexed by a non-negative tuning parameter α. For each value of α there corresponds a subtree $T \subset T_0$ such that

$$\sum_{m=1}^{|T|} \sum_{i:\, x_i \in R_m} (y_i - \hat{y}_{R_m})^2 + \alpha |T|$$

is as small as possible. Here $|T|$ indicates the number of terminal nodes.

Akin to LASSO, the tuning parameter α controls a trade-off between the subtree's complexity and its fit to the training data.

Such an α can be chosen using cross-validation.



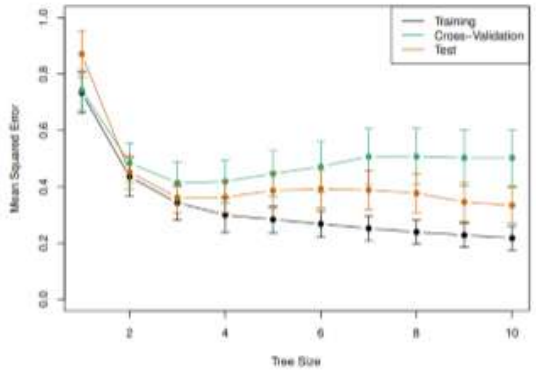


FIGURE 8.5. *Regression tree analysis for the* Hitters *data. The training, cross-validation, and test MSE are shown as a function of the number of terminal nodes in the pruned tree. Standard error bands are displayed. The minimum cross-validation error occurs at a tree size of three.*

Classification trees

A classification tree is very similar to a regression tree, except that it is tree used to predict a *qualitative* response rather than a quantitative one.

In contrast, for a classification tree, we predict that each observation belongs to the most commonly occurring class of training observations in the region to which it belongs.

In interpreting the results of a classification tree, we are often interested not only in the *class prediction* corresponding to a particular terminal node region, but also in the *class proportions* among the training observations that fall into that region.

Just as in the regression setting, we use recursive binary splitting to grow a classification tree. However, in the classification setting, *RSS* cannot be used as a criterion for making the binary splits.

A natural alternative to RSS is the classification error rate.

Splitting criterion

The most common measure is to minimize Gini index defined by

$$G = \sum_{k=1}^{K} \hat{p}_{mk}(1 - \hat{p}_{mk}),$$

which is a measure of total variance across the K classes.

Here $\hat{p}_{mk}$ represents the proportion of training observations in the mth region that are from the kth class.

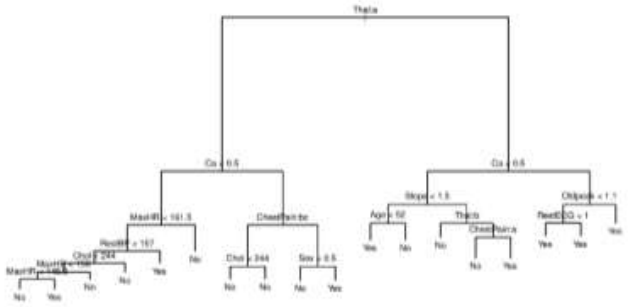
Gini index takes on a small value if all of the $\hat{p}_{mk}$'s are close to zero or one.

For this reason the Gini index is referred to as a measure of *node purity*—a small value indicates that a node contains predominantly observations from a single class.

Other measures such as classification error rate and entropy can also be used.

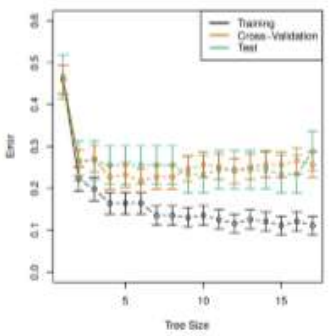
An example

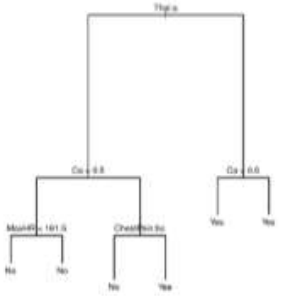
Following shows an example on the *Heart* data set. These data contain a binary outcome *HD* for 303 patients who presented with chest pain. An outcome value of *Yes* indicates the presence of heart disease based on an angiographic test, while *No* means no heart disease.



An example

Following shows an example on the *Heart* data set. These data contain a binary outcome *HD* for 303 patients who presented with chest pain. An outcome value of *Yes* indicates the presence of heart disease based on an angiographic test, while *No* means no heart disease.





A surprising characteristic

Some splits yield two terminal nodes that have the same predicted value. For instance, consider the split *RestECG*< 1 near the bottom right of the unpruned tree. Regardless of the value of *RestECG*, a response Yes is predicted for those observations. Why, then, is the split performed at all?

A surprising characteristic

Some splits yield two terminal nodes that have the same predicted value. For instance, consider the split *RestECG*< 1 near the bottom right of the unpruned tree. Regardless of the value of *RestECG*, a response Yes is predicted for those observations. Why, then, is the split performed at all?

The split is performed because it leads to increased node purity.

That is, all 9 of the observations corresponding to the right-hand leaf have a response value of Yes, whereas 7/11 of those corresponding to the left-hand leaf have a response value of Yes.
Why is node purity important?

A surprising characteristic

Some splits yield two terminal nodes that have the same predicted value. For instance, consider the split *RestECG*< 1 near the bottom right of the unpruned tree. Regardless of the value of *RestECG*, a response Yes is predicted for those observations. Why, then, is the split performed at all?

The split is performed because it leads to increased node purity.

That is, all 9 of the observations corresponding to the right-hand leaf have a response value of Yes, whereas $7/11$ of those corresponding to the left-hand leaf have a response value of Yes.
Why is node purity important?

Suppose that we have a test observation that belongs to the region given by that right-hand leaf. Then we can be pretty certain that its response value is Yes. In contrast, if a test observation belongs to the region given by the left-hand leaf, then its response value is probably Yes, but we are much less certain.

Trees Versus Linear Models

In particular, linear regression assumes a model of the form

$$f(X) = \beta_0 + \sum_{j=1}^{p} X_j \beta_j,$$

whereas regression trees assume a model of the form

$$f(X) = \sum_{m=1}^{M} c_m \cdot 1_{(X \in R_m)}$$

Which model is better? It depends on the problem at hand.

If the relationship between the features and the response is well approximated by a linear model, then linear regression will likely outperform regression tree.

If instead there is a highly nonlinear and complex relationship between the features and the response, then decision trees may outperform classical approaches.

Trees Versus Linear Models

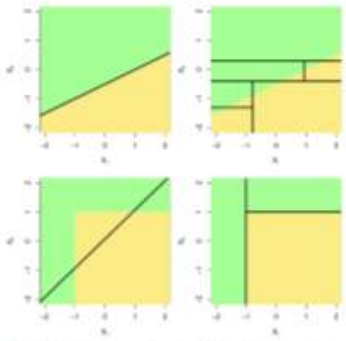


FIGURE 8.7. Top Row: *A two-dimensional classification example in which the true decision boundary is linear, and is indicated by the shaded regions. A classical approach that assumes a linear boundary (left) will outperform a decision tree that performs splits parallel to the axes (right).* Bottom Row: *Here the true decision boundary is non-linear. Here a linear model is unable to capture the true decision boundary (left), whereas a decision tree is successful (right).*

21 / 30

Advantages and Disadvantages of Trees

- ↑ Trees are very easy to explain to people. In fact, they are even easier to explain than linear regression!
- ↑ Some people believe that decision trees more closely mirror human decision-making than do the regression and classification approaches.
- ↑ Trees can be displayed graphically, and are easily interpreted even by a non-expert (especially if they are small).
- ↑ Trees can easily handle qualitative predictors without the need to create dummy variables.
- ↓ Unfortunately, trees generally do not have the same level of predictive accuracy as some of the other regression and classification approaches.
- ↓ Additionally, trees can be very non-robust. In other words, a small change in the data can cause a large change in the final estimated tree.

22 / 30

Ensemble methods

An ensemble method combines many simple "building ensemble block" models in order to obtain a single and potentially very powerful model.

These simple building block models are sometimes known as *weak learners*, since they may lead to mediocre predictions on their own.

The decision trees discussed so far suffer from *high variance*.

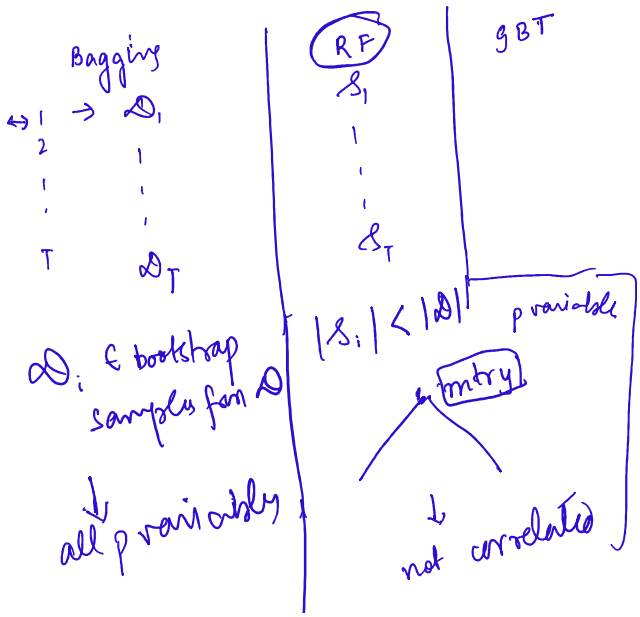
If we split the training data into two parts at random, and fit a decision tree to both halves, the results that we get could be quite different.

In contrast, a procedure with low variance will yield similar results if applied repeatedly to distinct data sets; linear regression tends to have low variance, if the ratio of n to p is moderately large.

Bootstrap aggregation, or *bagging*, is a general-purpose procedure for reducing the bagging variance of a statistical learning method.

Bootstrap training set and take avg of estimators obtained across bootstrap samples.

23 / 30



reducing the bagging variance of a statistical learning method.

Bootstrap training set and take avg of estimators obtained across bootstrap samples.

23 / 30

Bagging

To apply bagging to regression trees, we simply construct B regression trees using B **bootstrapped** training sets, and average the resulting predictions.

These trees are grown deep, and are not pruned. Hence each individual tree has high variance, but low bias.

Averaging these B trees reduces the variance.

Bagging has been demonstrated to give impressive improvements in accuracy by combining together hundreds or even thousands of trees into a single procedure.

For classification problems, one way to combine predictions is to take a *majority vote*.

For a given test observation, we can record the class predicted by each of the B trees, and take a majority vote: the overall prediction is the most commonly occurring majority vote class among the B predictions.

24 / 30

Variable Importance Measures

Bagging improves prediction accuracy at the expense of interpretability.

When we bag a large number of trees, it is no longer possible to represent the resulting statistical learning procedure using a single tree, and it is no longer clear which variables are most important to the procedure.

One can obtain an overall summary of the importance of each predictor using the RSS (for bagging regression trees) or the Gini index (for bagging classification trees).

In the case of bagging regression trees, we can record the total amount that the RSS is decreased due to splits over a given predictor, averaged over all B trees. A large value indicates an important predictor.

Similarly, in the context of bagging classification trees, we can add up the total amount that the Gini index is decreased by splits over a given predictor, averaged over all B trees.

25 / 30

Variable importance

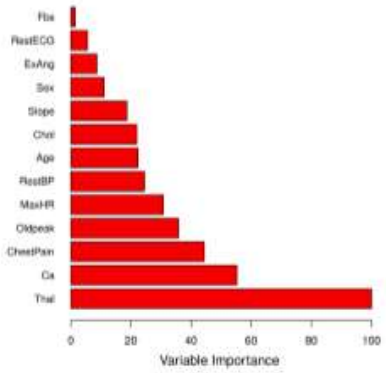


FIGURE 8.9. *A variable importance plot for the* Heart *data. Variable importance is computed using the mean decrease in Gini index, and expressed relative to the maximum.*

Random Forests

Random forests provide an improvement over bagged trees by way of a random forest small tweak that decorrelates the trees.

As in bagging, we build a number of decision trees on bootstrapped training samples.

But when building these decision trees, each time a split in a tree is considered, a random sample of m predictors is chosen as split candidates from the full set of p predictors.

The split is allowed to use only one of those m predictors.

A fresh sample of m predictors is taken at each split, and typically we choose $m = \sqrt{p}$—that is, the number of predictors considered at each split is approximately equal to the square root of the total number of predictors.

May sound crazy but has a clever rationale.

Random Forests

Suppose that there is one very strong predictor in the data set, along with a number of other moderately strong predictors. Then, in the collection of bagged trees, most or all of the trees will use this strong predictor in the top split.

Consequently, all of the bagged trees will look quite similar to each other.

Hence the predictions from the bagged trees will be highly correlated.

Unfortunately, averaging many highly correlated quantities does not lead to as large of a reduction in variance as averaging many uncorrelated ones.

In particular, this means that bagging will not lead to a substantial reduction in variance over a single tree in this setting.

Random forests overcome this problem by forcing each split to consider only a subset of the predictors. We can think of this process as *decorrelating* the trees, thereby making the average of the resulting trees less variable and hence more reliable.

Impact of *mtry*

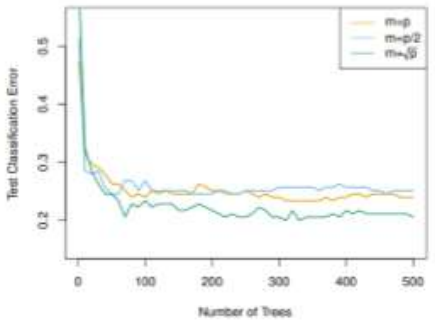


FIGURE 8.10. *Results from random forests for the 15-class gene expression data set with p = 500 predictors. The test error is displayed as a function of the number of trees. Each colored line corresponds to a different value of m, the number of predictors available for splitting at each interior tree node. Random forests (m < p) lead to a slight improvement over bagging (m = p). A single classification tree has an error rate of 45.7%.*

29 / 30

Performance

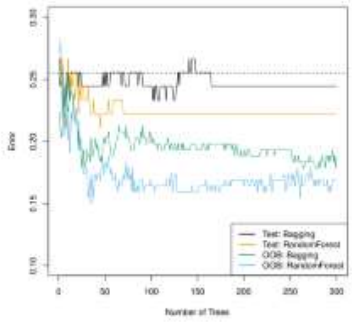
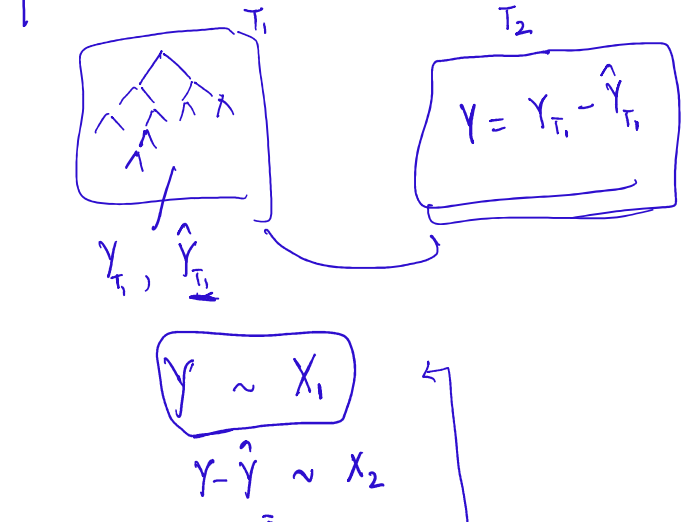


FIGURE 8.8. *Bagging and random forest results for the Heart data. The test error (black and orange) is shown as a function of B, the number of bootstrapped training sets used. Random forests were applied with $m = \sqrt{p}$. The dashed line indicates the test error resulting from a single classification tree. The green and blue traces show the OOB error, which in this case is — by chance — considerably lower.*

30 / 30

Gradient Boosted Trees



$y - \hat{y} \sim x_2$

$y - \hat{\hat{y}} \sim x_3$